

UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
DEPARTAMENTO DE COMPUTACIÓN

PRÁCTICA DE LABORATORIO N° 4

Buffer Circular con Interrupciones de Teclado y Reloj y Semáforo con Pulsador e Interrupciones

Autores:

Genesis V. Cabello M.
C.I:30.367.386

Maivet S. Pereira M.
C.I: 30.814.708

Profesor:

Jose Canache

Asignatura:

Arquitectura del Computador

Objetivo

Familiarizar al estudiante con la sintaxis del lenguaje ensamblador MIPS32 y su ejecución en el entorno MARS, resolviendo problemas clásicos de programación, prestando especial atención al uso de interrupciones para la gestión de entrada/salida (E/S) y la utilización de rutinas de servicio para responder a eventos como entrada por teclado y control de tiempo, simulando el funcionamiento de periféricos.

1. Investigación Preliminar

- 1.1. Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware (desde que ocurre el evento hasta que se atiende).



Figura 1: diagrama

- 1.2. ¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?

Sondeo (Polling): El procesador ejecuta de forma sincrónica un bucle continuo leyendo el registro de estado del periférico (lw) hasta que un bit específico indica que el dispositivo está listo. El control del tiempo lo mantiene en su totalidad la CPU.

Interrupciones: La gestión es asíncrona. El procesador le indica al periférico qué acción realizar y continúa ejecutando otras tareas de manera independiente. Cuando el periférico completa su operación o recibe datos (por ejemplo, al presionar la tecla 's'), envía una señal eléctrica a la CPU para pausar temporalmente el programa principal y atender la E/S.

- 1.3. ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?

Eliminación de esperas activas: Libera el 100% de los ciclos de reloj que la CPU desperdiciaría en bucles de consulta, permitiendo procesar otros algoritmos o entrar en estados de bajo consumo.

Respuesta en tiempo real: Los eventos externos críticos se atienden con una latencia mínima en el instante exacto en que ocurren, garantizando la reactividad del sistema.

Eficiencia multitarea: Facilita la ejecución concurrente de eventos independientes, como temporizadores y lecturas de teclado simultáneas.

1.4. ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?

Las interrupciones en MIPS32 son administradas por el Coprocesador 0 (CP0) mediante los siguientes registros clave:

Status (Registro \$12): Mantiene la configuración de control. Contiene el bit global de habilitación de interrupciones (IE, bit 0) y la máscara de interrupciones (IM, bits 8 a 15) para habilitar o bloquear fuentes individuales.

Cause (Registro \$13): Registra el origen o motivo del evento. Sus bits IP (Pending Interrupts, bits 8 a 15) indican qué línea de interrupción solicitó atención, y el campo ExcCode (bits 2 a 6) identifica si fue una interrupción o una excepción interna.

EPC (Exception Program Counter, Registro \$14): Almacena la dirección de la instrucción que fue interrumpida para poder reanudar el flujo más adelante.

BadVAddr (Registro \$8): Contiene la dirección de memoria virtual que causó un fallo en caso de excepciones de memoria.

1.5. ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?

Dado que las interrupciones ocurren de forma asíncrona e impredecible, el programa principal desconoce cuándo será interrumpido. La Rutina de Servicio a Interrupción (ISR) requiere utilizar registros generales (\$at, \$v0, \$a0, \$t0-\$t9) para ejecutar su propia lógica. Si la ISR modifica estos registros sin salvar su valor previo en la pila, al retornar mediante `eret`, el programa principal continuará su ejecución con datos corrompidos en sus registros de trabajo, causando fallos graves de cálculo o saltos erróneos.

1.6. Momentos en que pueden generarse excepciones en un sistema MIPS32. a) Enumera al menos 4 situaciones en las que se pueda generar una excepción (por ejemplo: desbordamiento aritmético, fallo de dirección, etc.). b) Explica qué etapas del pipeline pueden provocar una excepción y por qué?.

a) Situaciones típicas de generación de excepciones

- Desbordamiento aritmético (Arithmetic Overflow): Al ejecutar operaciones como add o sub cuyo resultado excede el rango representable en complemento a dos.
- Error de dirección / Alineación (Address Error): Intentar leer o escribir una palabra de 32 bits (lw/sw) en una dirección que no es múltiplo de 4.
- Instrucción no válida / reservada (Reserved Instruction): Intentar ejecutar un código de operación (opcode) no definido en el repertorio MIPS32.
- Llamada explícita al sistema (Syscall) o punto de interrupción (Breakpoint): Generadas intencionalmente por software para solicitar servicios del kernel.

b) Etapas del pipeline que pueden provocar una excepción

- IF (Instruction Fetch): Error de dirección al intentar buscar una instrucción en un PC no alineado o fuera de la memoria permitida.
- ID (Instruction Decode): Detección de una instrucción no definida o un intento de acceder al Coprocesador 0 en modo Usuario.
- EX (Execution): Generación de desbordamiento aritmético en la ALU o división por cero.
- MEM (Memory Access): Violación de permisos de acceso a páginas de memoria protegidas o direcciones de datos no alineadas.

1.7. Estrategias de tratamiento de excepciones e interrupciones

a) Explica las diferencias entre interrupciones y excepciones (¿son síncronas o asíncronas?). b) Describe brevemente dos estrategias para tratar excepciones en un sistema MIPS32¿Cómo se redirige la ejecución hacia la rutina de servicio? ¿Cuál es la función del registro EPC (Exception Program Counter)?

a) Diferencias entre interrupciones y excepciones

- Interrupciones: Son asíncronas. Son provocadas por eventos del hardware externos al núcleo del procesador (como la llegada de un carácter al teclado o la expiración de un reloj). Ocurren independientemente de la instrucción que se esté ejecutando.
- Excepciones: Son síncronas. Ocurren como consecuencia directa e interna de la ejecución de una instrucción particular de la CPU (como una división por cero o un fallo de página).

b) Tratamiento, redirección y función del registro EPC

- Redirección: Cuando se detecta una excepción o interrupción, la lógica de control del hardware fuerza de inmediato el PC a una dirección fija en memoria codificada en el procesador (en MIPS32, comúnmente 0x80000180 en el segmento .ktext).

- Función del EPC: El hardware copia la dirección de la instrucción interrumpida en el registro EPC. En excepciones síncronas, almacena la dirección de la instrucción defectuosa; en interrupciones asíncronas, almacena la dirección de la instrucción que se ejecutaría a continuación. Al finalizar el tratamiento, la instrucción de retorno eret lee el registro EPC y restaura el control al código principal.

1.8. Habilitación de interrupciones en dispositivos y procesador

a) Explica cómo se habilitan las interrupciones en: El teclado. La pantalla. El procesador (detalla qué bits del registro Status deben modificarse). b) ¿Qué pasaría si habilitamos interrupciones en los dispositivos, pero no en el procesador?

a) Procedimiento de habilitación

- Teclado: Se debe poner a 1 el bit 1 (Interrupt Enable) del registro de control del receptor del teclado mapeado en la memoria (dirección 0xFFFF0000 en MARS).
- Pantalla: Se debe poner a 1 el bit 1 (Interrupt Enable) del registro de control del transmisor de la pantalla (dirección 0xFFFF0008 en MARS).
- Procesador: Se configura el registro \$12 (Status de CP0) escribiendo un 1 en el bit 0 (IE - Interrupt Enable global) y activando los bits 8 al 15 (IM - Interrupt Mask) según la línea de interrupción específica requerida (por ejemplo, el bit de interrupción por hardware del temporizador o teclado).

b) Dispositivos habilitados pero procesador no habilitado

Si el periférico tiene sus interrupciones locales habilitadas pero el procesador mantiene su bit global IE en 0 o la máscara IM desactivada, las señales de solicitud de interrupción sentadas en el bus por el dispositivo serán totalmente ignoradas (enmascaradas) por la CPU. El hardware no pausará la ejecución ni saltará a la rutina de servicio.

1.9. Procesamiento de interrupciones a) Describe paso a paso qué ocurre cuando se produce una interrupción de reloj: Desde que el evento ocurre hasta que la rutina de servicio termina. ¿Qué registros se guardan y restauran? ¿Qué hace el hardware y qué hace el software (sistema operativo o rutina)? b) ¿Por qué es importante guardar el contexto (registros generales, EPC, Status) al entrar en la rutina?

a) Paso a paso ante una interrupción de reloj

- Ocurrencia del evento (Hardware): El temporizador de hardware alcanza la cuenta correspondiente a los segundos programados y eleva su señal de E/S.
- Interrupción (Hardware): La CPU detecta la línea activa. Al finalizar el ciclo de reloj actual, guarda el PC en EPC, registra la causa en Cause, pone el bit IE de Status a 0 (deshabilitando otras interrupciones) y actualiza el PC con la dirección 0x80000180.

- Guardado de contexto (Software/ISR): La ISR reserva espacio en la pila mediante la manipulación de registros y almacena \$at, \$v0, \$a0, etc., para garantizar su preservación.
- Atención a la interrupción (Software/ISR): La ISR consulta Cause para confirmar que se trata de la interrupción de reloj, borra la señal de interrupción del temporizador y ejecuta la actualización de estado del sistema (por ejemplo, cambiar el estado del semáforo).
- Restauración y retorno (Software/Hardware): La ISR recupera los registros guardados en la pila, rehabilita el bit IE en Status y ejecuta la instrucción eret, haciendo que el hardware copie el contenido de EPC de regreso al PC.

b) Importancia de guardar el contexto

El contexto incluye los registros generales de trabajo, el EPC y el registro Status. Guardarlos es crítico porque una interrupción no es anticipada por la aplicación en ejecución. Si la ISR altera cualquiera de estos registros sin restituirlos punto por punto, se corromperán datos, se perderá la dirección de retorno original o se modificará el nivel de privilegios del procesador.

1.10. Interrupciones de reloj y control de ejecución a) Explica cómo una interrupción de reloj puede usarse para: Evitar que un programa quede en un bucle infinito. Finalizar programas que superan un tiempo máximo de ejecución. b) ¿Qué debe hacer el software si el programa finaliza antes de que ocurra la interrupción de reloj?

a) Control de bucles infinitos y tiempos de ejecución (Timeout) El temporizador de hardware opera mediante una escala de tiempo independiente del flujo de instrucciones de la CPU. Al cargar un valor de temporización (como 20 segundos para la captura en el buffer de teclado), la interrupción de reloj actuará como un watchdog timer. Cuando el tiempo expira, la CPU interrumpe de forma obligatoria el flujo del programa, sin importar si este se encuentra atrapado en un bucle infinito de usuario o esperando datos, cediendo el control a la ISR para vaciar el buffer, forzar la salida o reestructurar el estado del sistema.

b) Procedimiento ante finalización anticipada del programa Si el programa concluye su tarea antes de que la interrupción del reloj se dispare, el software debe cancelar explícitamente el temporizador limpiando sus registros de control local y restaurando la máscara del registro Status en el CP0. De no hacerse, el temporizador continuaría contando en segundo plano e interrumpiría el procesador cuando este ya esté ejecutando una tarea no relacionada, generando una interrupción spuria o invadiendo memoria no asignada.

1.11. Análisis y Discusión de los Resultados

La realización de los dos ejercicios propuestos en esta práctica evidenció de manera clara las ventajas del modelo de procesamiento impulsado por eventos:

Buffer de Teclado con Filtro de Caracteres: La implementación del almacenamiento en un buffer circular durante intervalos rígidos de 20 segundos demostró la utilidad de combinar interrupciones de teclado (para la captura e inspección en tiempo real de letras mayúsculas A- Z) con interrupciones de reloj (encargadas de marcar los 20 segundos para la descarga e impresión). Se constató la efectividad de las interrupciones para filtrar caracteres inválidos sobre la marcha sin ralentizar la ejecución.

Semáforo con Pulsador: La simulación de la máquina de estados del semáforo evidenció cómo la combinación de eventos de entrada por teclado (pulsación de la tecla 39;s39;) e interrupciones diferidas por temporizador (secuencias temporizadas de 20, 10 y 30 segundos) permite construir sistemas concurrentes y reactivos de bajo nivel sin bloquear los recursos del procesador.

Los resultados confirman que el uso de interrupciones, la gestión adecuada del Coprocesador 0 y el estricto resguardo del contexto en la pila resultan indispensables para diseñar sistemas embebidos estructurados, eficientes y fiables en arquitecturas MIPS32.